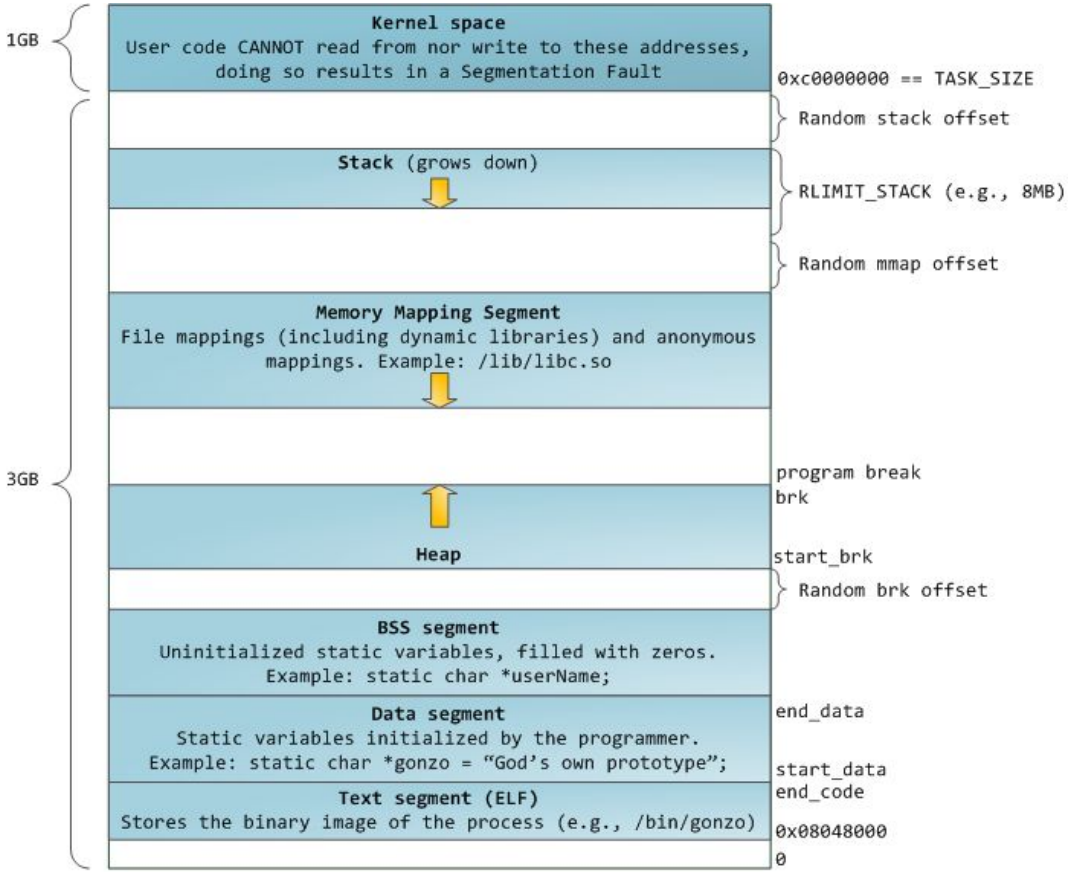
CSE 4351: Computer Security



Instructor: Dr. MD Armanuzzaman

Memory Map of Linux Process (32 bit system)



https://manybutfinite.com/post/anatomy-of-a-program-in-memory/

The Heap

The heap is pool of memory used for dynamic allocations at runtime. Heap memory is different from stack memory in that it is ***persistent between functions***.

– **malloc**() grabs memory on the heap; keyword ***new*** in C++

– **free**() releases memory on the heap; keyword ***delete*** in C++

Both are standard C library interfaces. Neither of them directly mapps to a system call.

malloc() and free()

```
void* malloc(size_t size);
```

Allocates size bytes of uninitialized storage. If allocation succeeds, returns a pointer that is suitably aligned for any object type with fundamental alignment.

```
void free(void* ptr);
```

Deallocates the space previously allocated by malloc(), etc.

calloc() and realloc()

```
void *calloc(size_t nitems, size_t size)
```

The difference in malloc and calloc is that malloc does not set the memory to zero whereas calloc sets allocated memory to zero.

```
void *realloc(void *ptr, size_t size)
```

Resize the memory block pointed to by ptr that was previously allocated with a call to malloc or calloc.

How to use malloc() and free()

```
int main()
{
        char * buffer = NULL;

        /* allocate a 0x100 byte buffer */
        buffer = malloc(0x100);

        /* read input and print it */
        fgets(stdin, buffer, 0x100);
        printf("Hello %s!\n", buffer);

        /* destroy our dynamically allocated buffer */
        free(buffer);
        return 0;
}
```

Heap vs. Stack

Heap

- Dynamic memory allocations at runtime
- Objects, big buffers, structs, persistence, larger things

Slower, Manual

- Done by the programmer
- malloc/calloc/recalloc/free
- new/delete

Stack

- Fixed memory allocations known at compile time
- Local variables, return addresses, function args

Fast, Automatic; Done by the compiler

- Abstracts away any concept of allocating/de-allocating

Heap Implementations

Doug Lea malloc or **dlmalloc**. Default native version of malloc in some old distributions of Linux (http://gee.cs.oswego.edu/dl/html/malloc.html)

ptmalloc. ptmalloc is based on dlmalloc and was extended for use with multiple threads. On Linux systems, ptmalloc has been put to work for years as part of the GNU C library.

tcmalloc. Google's customized implementation of C's malloc() and C++'s operator new (https://github.com/google/tcmalloc)

jemalloc. jemalloc is a general purpose malloc(3) implementation that emphasizes fragmentation avoidance and scalable concurrency support.

The **Hoard** memory allocator. UMass Amherst CS Professor Emery Berger

Which implementation on our server?

ldd --version

GLIBC 2.31, Ptmalloc2

https://elixir.bootlin.com/glibc/glibc-2.31/source/malloc/malloc.c

```
ctf@heapexploitation_heapchunks_32:/$ ldd --version
ldd (Ubuntu GLIBC 2.31-0ubuntu9.2) 2.31
Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
Written by Roland McGrath and Ulrich Drepper.
```

Disclaimer: Ptmalloc is very complex, and its implementation is constantly changing.

This is an approximation to glibc 2.31

Overview of dlmalloc

The Linux version of the dynamic memory allocator. Even though it has been updated, from the point of view of software infused bugs and exploits, new versions are still more or less similar to the original one.

Design goals:

Maximizing Portability To rely on as few system-dependent features as possible, system calls in particular.

Minimizing Space The allocator should not waste memory. It should obtain the least amount of memory from the system it requires, and should maintain memory in ways that minimize fragmentation—that is, it should try to avoid creating a large number of contiguous chunks of memory that are not used by the program.

Minimizing Time The malloc(), free(), and realloc calls should be fast on average.

Overview of dlmalloc

The Linux version of the dynamic memory allocator. Even though it has been updated, from the point of view of software infused bugs and exploits, new versions are still more or less similar to the original one.

Design goals:

Maximizing Locality Allocate chunks of memory that are typically requested or used together near each other. This will help minimize CPU page and cache misses.

Maximizing Error Detection Should provide some means for detecting corruption due to overwriting memory, multiple frees, and so on. It is not supposed to work as a general memory leak detection tool at the cost of slowing down.

Minimizing Anomalies It should have reasonably similar performance characteristics across a wide range of possible applications whether they are GUI or server programs, string processing applications, or network tools.

Malloc_chunk (ptmalloc2 in glibc2.31)

```
struct malloc_chunk {

  INTERNAL_SIZE_T      mchunk_prev_size;  /* Size of previous chunk (if free).  */
  INTERNAL_SIZE_T      mchunk_size;       /* Size in bytes, including overhead. */

  struct malloc_chunk* fd;         /* double links -- used only if free. */
  struct malloc_chunk* bk;

  /* Only used for large blocks: pointer to next larger size.  */
  struct malloc_chunk* fd_nextsize; /* double links -- used only if free. */
  struct malloc_chunk* bk_nextsize;
};
```

For both in-use and freed

Only for freed

The freed chunks are double-linked

INTERNAL_SIZE_T is the same as size_t. 8 bytes in 64 bit;
4 bytes in 32 bits machine.
Pointer is 8/4 bytes on a 64/32 bit machine, respectively.

Alignment is defined as 2 * (sizeof(size_t))

https://elixir.bootlin.com/glibc/glibc-2.31/source/malloc/malloc.c

Heap Chunks (figures in 32 bit)

buffer = malloc(0x100);

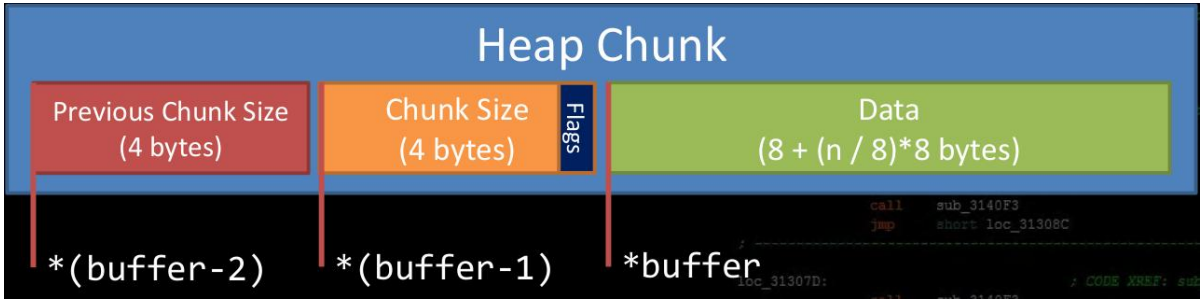
//Out comes a heap chunk

Previous Chunk Size: Size of previous chunk (if prev chunk is free)
Chunk Size: Size of entire chunk including overhead
Data: Your newly allocated memory / ptr returned by malloc
Flags: Because of byte alignment, the lower 3 bits of the chunk size field would always be zero. Instead they are used for flag bits.

0x01 PREV_INUSE – set when previous chunk is in use

0x02 IS_MMAPPED – set if chunk was obtained with mmap()

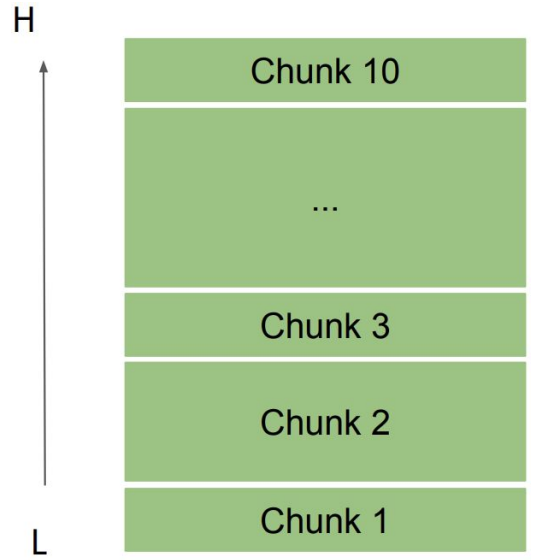
0x04 NON_MAIN_ARENA – set if chunk belongs to a thread arena



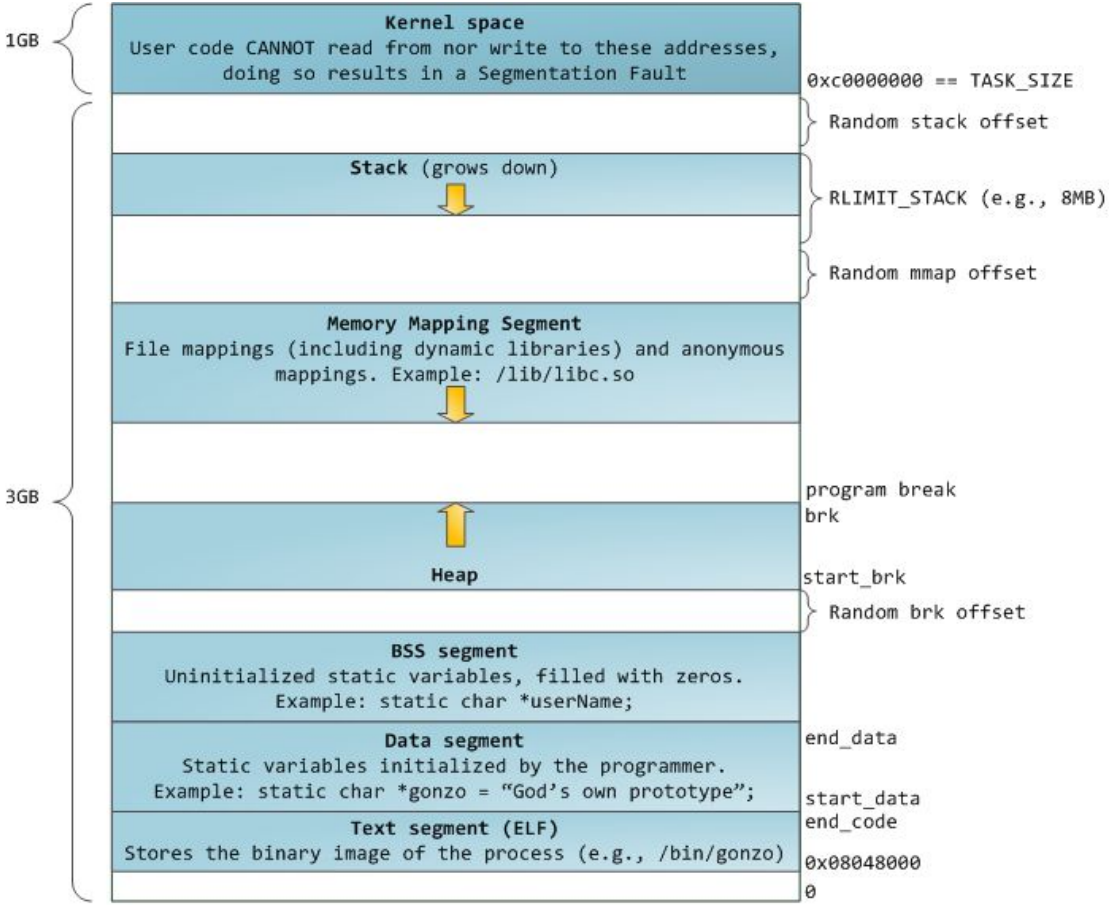
code/heapsizes

```
int main()
{
        unsigned int lengths[] = {32, 4, 20, 0, 64, 32, 32, 32, 32, 32};
        unsigned int * ptr[10];

        int i;
        for(i = 0; i < 10; i++)
                ptr[i] = malloc(lengths[i]);
        for(i = 0; i < 9; i++)
                printf("malloc(%2d) is at 0x%08x, %3d bytes to the next
        pointer\n",
                lengths[i],
                (unsigned int)ptr[i],
                (ptr[i+1]-ptr[i])*sizeof(unsigned int));
        return 0;
}
```



Heap goes from **low** address to **high** address



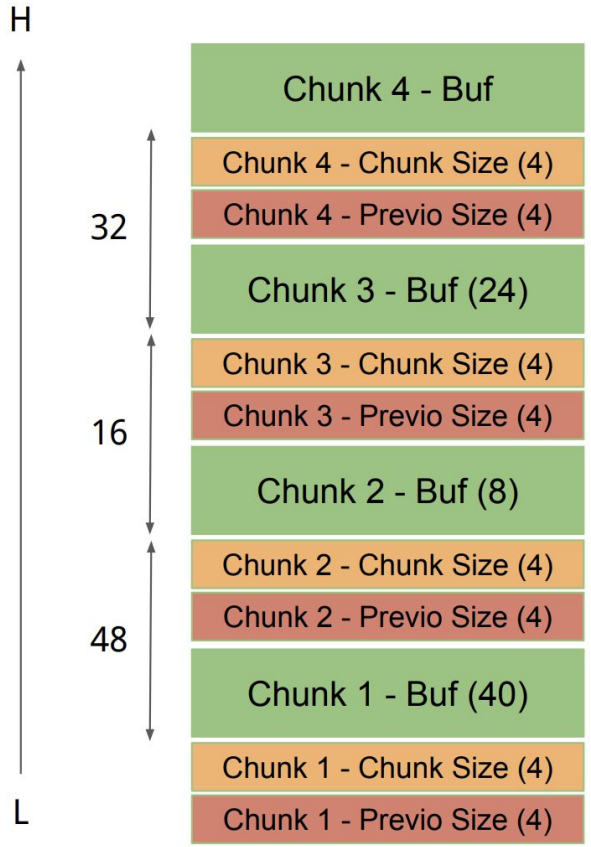
https://manybutfinite.com/post/anatomy-of-a-program-in-memory/

code/heapsizes 32bit

chunk_sizes = {32, 4, 20, 0, 64, 32, 32, 32, 32, 32};

```
ctf@heapexploitation_heapsizes_32:/$ ./heapexploitation_heapsizes_32
The size of size_t is 4
The size of a pointer is 4
malloc(32) is at 0x5655a5b0,  48 bytes to the next pointer
malloc( 4) is at 0x5655a5e0,  16 bytes to the next pointer
malloc(20) is at 0x5655a5f0,  32 bytes to the next pointer
malloc( 0) is at 0x5655a610,  16 bytes to the next pointer
malloc(64) is at 0x5655a620,  80 bytes to the next pointer
malloc(32) is at 0x5655a670,  48 bytes to the next pointer
malloc(32) is at 0x5655a6a0,  48 bytes to the next pointer
malloc(32) is at 0x5655a6d0,  48 bytes to the next pointer
malloc(32) is at 0x5655a700,  48 bytes to the next pointer
```

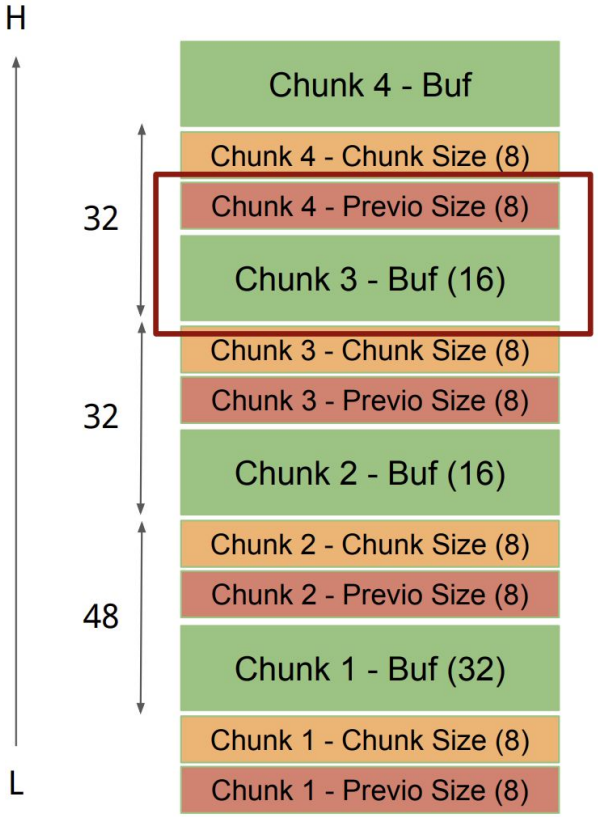
Alignment is at least defined as 2 * (sizeof(size_t)) = 8



code/heapsizes 64bit

chunk_sizes = {32, 4, **20**, 0, 64, 32, 32, 32, 32, 32};

```
ctf@heapexploitation_heapsizes_64:/$ ./heapexploitation_heapsizes_64
The size of size_t is 8
The size of a pointer is 8
malloc(32) is at 0x555596b0,  48 bytes to the next pointer
malloc( 4) is at 0x555596e0,  32 bytes to the next pointer
malloc(20) is at 0x55559700,  32 bytes to the next pointer
malloc( 0) is at 0x55559720,  32 bytes to the next pointer
malloc(64) is at 0x55559740,  80 bytes to the next pointer
malloc(32) is at 0x55559790,  48 bytes to the next pointer
malloc(32) is at 0x555597c0,  48 bytes to the next pointer
malloc(32) is at 0x555597f0,  48 bytes to the next pointer
malloc(32) is at 0x55559820,  48 bytes to the next pointer
```



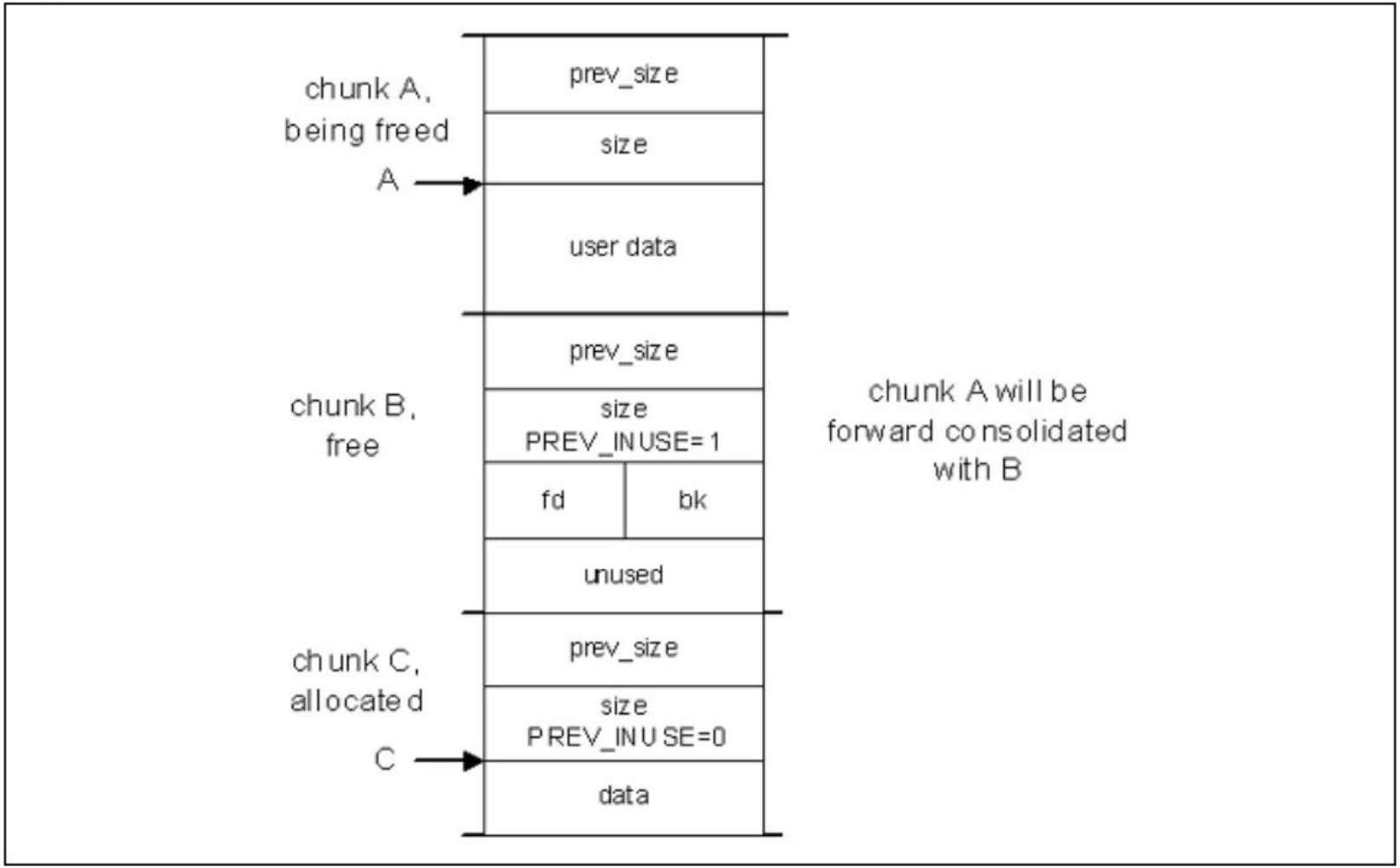
Alignment is defined as 2 * (sizeof(size_t)) = 16

Chunk4's previous size field is used by Chunk3

Malloc Trivia

How many bytes on the heap are your ***malloc chunks*** really taking up?

- malloc(32); 48 bytes (32bit/64bit)
- malloc(4); 16 bytes (32bit) / 32 bytes (64bit)
- malloc(20); 32 bytes (32bit/64bit [Prev Size field reused])
- malloc(0); 16 bytes (32bit) / 32 bytes (64bit)


chunk A,
being freed
A
prev_size
size
user data
chunk B,
free
prev_size
size
PREV_INUSE=1
fd
bk
unused
chunk C,
allocated
C
prev_size
size
PREV_INUSE=0
data
chunk A will be
forward consolidated
with B

code/malloc_chunks

```
void print_chunk(size_t * ptr, unsigned int len)
{
    printf("[prev - 0x%016x][size - 0x%08x][buffer (0x%016x)] - from malloc(%d)\n", *(ptr-2), *(ptr-1),
(unsigned int)ptr, len); }

int main()
{
   void * ptr[LEN];
   unsigned int lengths[] = {0, 4, 8, 16, 24, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384};
   int i;

   printf("mallocing...\n");

   for(i = 0; i < LEN; i++)
          ptr[i] = malloc(lengths[i]);

   for(i = 0; i < LEN; i++)
          print_chunk(ptr[i], lengths[i]);

 return 0;}
```

Modified from: https://github.com/RPISEC/MBE/blob/master/src/lecture/heap/heap_chunks.c

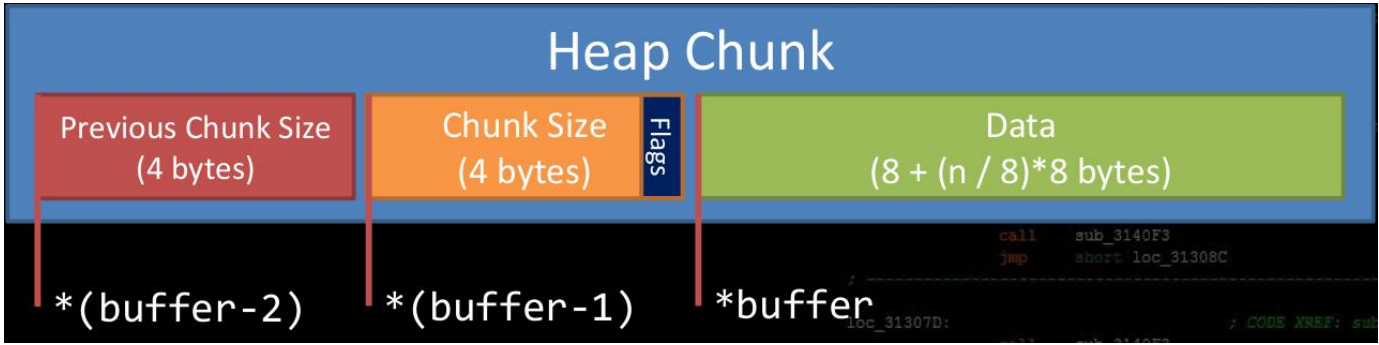
```
ctf@heapexploitation_malloc_chunks_32:/$ ./heapexploitation_malloc_chunks_32
mallocing...
[prev - 0x0000000000000000][size - 0x00000011][buffer (0x000000005655a5b0)] - from malloc(0)
[prev - 0x0000000000000000][size - 0x00000011][buffer (0x000000005655a5c0)] - from malloc(4)
[prev - 0x0000000000000000][size - 0x00000011][buffer (0x000000005655a5d0)] - from malloc(8)
[prev - 0x0000000000000000][size - 0x00000021][buffer (0x000000005655a5e0)] - from malloc(16)
[prev - 0x0000000000000000][size - 0x00000021][buffer (0x000000005655a600)] - from malloc(24)
[prev - 0x0000000000000000][size - 0x00000031][buffer (0x000000005655a620)] - from malloc(32)
[prev - 0x0000000000000000][size - 0x00000051][buffer (0x000000005655a650)] - from malloc(64)
[prev - 0x0000000000000000][size - 0x00000091][buffer (0x000000005655a6a0)] - from malloc(128)
[prev - 0x0000000000000000][size - 0x00000111][buffer (0x000000005655a730)] - from malloc(256)
[prev - 0x0000000000000000][size - 0x00000211][buffer (0x000000005655a840)] - from malloc(512)
[prev - 0x0000000000000000][size - 0x00000411][buffer (0x000000005655aa50)] - from malloc(1024)
[prev - 0x0000000000000000][size - 0x00000811][buffer (0x000000005655ae60)] - from malloc(2048)
[prev - 0x0000000000000000][size - 0x00001011][buffer (0x000000005655b670)] - from malloc(4096)
[prev - 0x0000000000000000][size - 0x00002011][buffer (0x000000005655c680)] - from malloc(8192)
[prev - 0x0000000000000000][size - 0x00004011][buffer (0x000000005655e690)] - from malloc(16384)
```

```
ctf@heapexploitation_malloc_chunks_64:/$ ./heapexploitation_malloc_chunks_64
mallocing...
[prev - 0x0000000000000000][size - 0x00000021][buffer (0x00000000555596b0)] - from malloc(0)
[prev - 0x0000000000000000][size - 0x00000021][buffer (0x00000000555596d0)] - from malloc(4)
[prev - 0x0000000000000000][size - 0x00000021][buffer (0x00000000555596f0)] - from malloc(8)
[prev - 0x0000000000000000][size - 0x00000021][buffer (0x0000000055559710)] - from malloc(16)
[prev - 0x0000000000000000][size - 0x00000021][buffer (0x0000000055559730)] - from malloc(24)
[prev - 0x0000000000000000][size - 0x00000031][buffer (0x0000000055559750)] - from malloc(32)
[prev - 0x0000000000000000][size - 0x00000051][buffer (0x0000000055559780)] - from malloc(64)
[prev - 0x0000000000000000][size - 0x00000091][buffer (0x00000000555597d0)] - from malloc(128)
[prev - 0x0000000000000000][size - 0x00000111][buffer (0x0000000055559860)] - from malloc(256)
[prev - 0x0000000000000000][size - 0x00000211][buffer (0x0000000055559970)] - from malloc(512)
[prev - 0x0000000000000000][size - 0x00000411][buffer (0x0000000055559b80)] - from malloc(1024)
[prev - 0x0000000000000000][size - 0x00000811][buffer (0x0000000055559f90)] - from malloc(2048)
[prev - 0x0000000000000000][size - 0x00001011][buffer (0x000000005555a7a0)] - from malloc(4096)
[prev - 0x0000000000000000][size - 0x00002011][buffer (0x000000005555b7b0)] - from malloc(8192)
[prev - 0x0000000000000000][size - 0x00004011][buffer (0x000000005555d7c0)] - from malloc(16384)
```

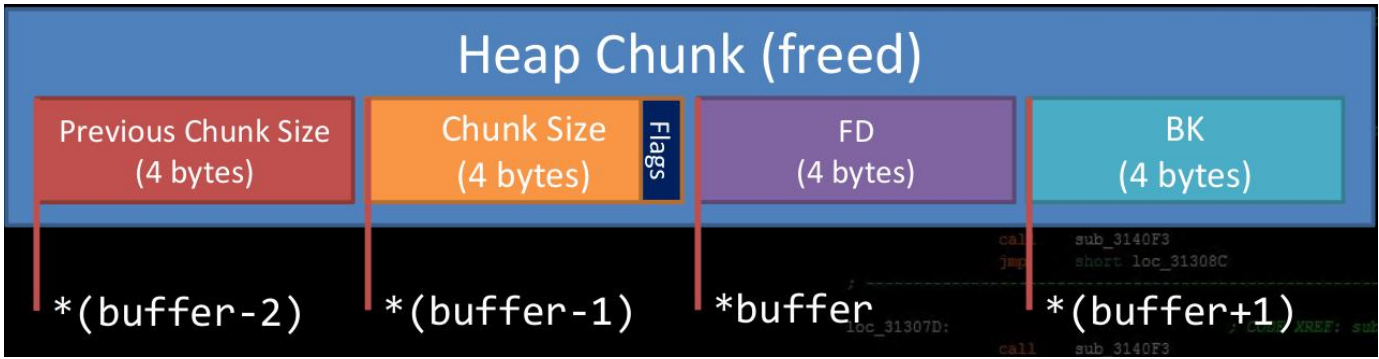
Heap Chunks – Two states (figures in 32 bit)

Heap chunks exist in two states

– in use (malloc'd)



– free'd.
Forward Pointer: A pointer to the next freed chunk
Backwards Pointer: A pointer to the previous freed chunk
Implementation-defined.



code/heap_frees

```
void print_inuse_chunk(size_t * ptr)
{
    printf("[prev - 0x%016x][size - 0x%016x][buffer (0x%016x)] -
Chunk 0x%016x - In use\n",
    *(ptr-2),
    *(ptr-1),
    (unsigned int)ptr,
    (unsigned int)(ptr-2));
}

void print_freed_chunk(size_t * ptr)
{
    printf("[prev - 0x%016x][size - 0x%016x][next - 0x%016x
    ][key - 0x%016x ] - Chunk 0x%016x - Freed\n",
    *(ptr-2),
    *(ptr-1),
    *ptr,
     *(ptr+1),
      (unsigned int)(ptr-2));
}
```

```
int main()
{
    size_t * ptr[LEN];
    unsigned int lengths[] = {32, 32, 32, 32, 32}; int i;
    printf("mallocing...\n");
    for(i = 0; i < LEN; i++)
            ptr[i] = malloc(lengths[i]);

    for(i = 0; i < LEN; i++)
            print_inuse_chunk(ptr[i]);

    printf("\nfreeing all chunks...\n");
    for(i = 0; i < LEN; i++)
            free(ptr[i]);
    for(i = 0; i < LEN; i++)
            print_freed_chunk(ptr[i]);

 return 0;}
```

Heap Overflow

- Buffer overflows are basically the same on the heap as they are on the stack
- Lots of cool and complex things like objects/structs end up on the heap
    - Anything that handles the data you just corrupted is now viable attack surface in the application
- It's common to put function pointers in structs which generally are malloc'd on the heap

code/heapoverflow1

```
void fly()
{
        printf("Flying ...\n");
}

typedef struct airplane
{
        void (*pfun)();
        char name[20];
} airplane
```

```
int main()
{
        printf("fly() at %p; print_flag() at %p\n", fly,
        print_flag);

        struct airplane *p1 = malloc(sizeof(airplane));
        printf("Airplane 1 is at %p\n", p1);

        struct airplane *p2 = malloc(sizeof(airplane));
        printf("Airplane 2 is at %p\n", p2);

        p1->pfun = fly;
        p2->pfun = fly;

        fgets(p2->name, 10, stdin);
        fgets(p1->name, 100, stdin);

        p1->pfun();
        p2->pfun();

        free(p1);
        free(p2);
        return 0;
}
```

code/heapoverflow1 64bit

```
void fly()
{
        printf("Flying ...\n");
}

typedef struct airplane
{
        void (*pfun)();
        char name[20];
} airplane
```

```
int main()
{
        printf("fly() at %p; print_flag() at %p\n", fly,
        print_flag);

        struct airplane *p1 = malloc(sizeof(airplane));
        printf("Airplane 1 is at %p\n", p1);

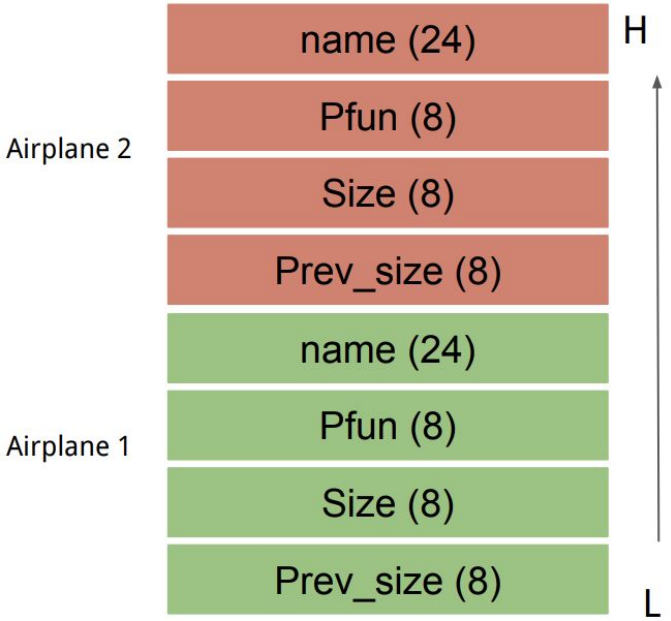
        struct airplane *p2 = malloc(sizeof(airplane));
        printf("Airplane 2 is at %p\n", p2);

        p1->pfun = fly;
        p2->pfun = fly;

        fgets(p2->name, 10, stdin);
        fgets(p1->name, 100, stdin);

        p1->pfun();
        p2->pfun();

        free(p1);
        free(p2);
        return 0;
}
```



code/heapoverflow1 64bit

```
void fly()
{
        printf("Flying ...\n");
}

typedef struct airplane
{
        void (*pfun)();
        char name[20];
} airplane
```

```
int main()
{
        printf("fly() at %p; print_flag() at %p\n", fly,
        print_flag);

        struct airplane *p1 = malloc(sizeof(airplane));
        printf("Airplane 1 is at %p\n", p1);

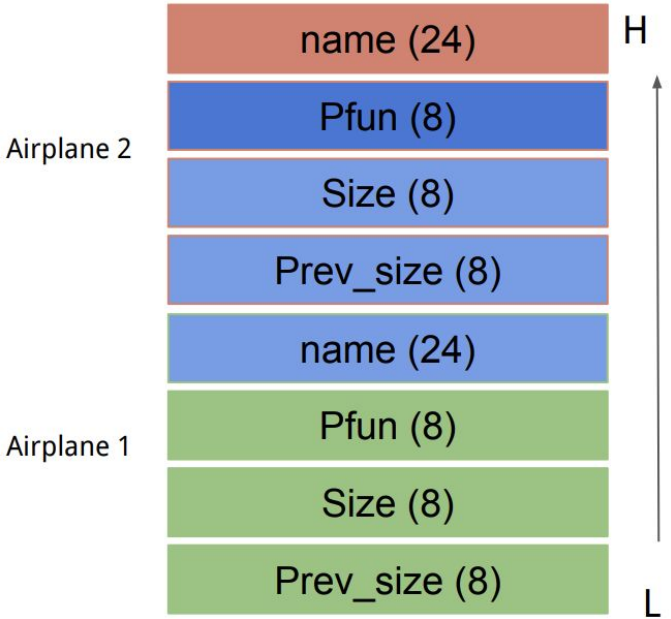
        struct airplane *p2 = malloc(sizeof(airplane));
        printf("Airplane 2 is at %p\n", p2);

        p1->pfun = fly;
        p2->pfun = fly;

        fgets(p2->name, 10, stdin);
        fgets(p1->name, 100, stdin);

        p1->pfun();
        p2->pfun();

        free(p1);
        free(p2);
        return 0;
}
```



Exploit looks like

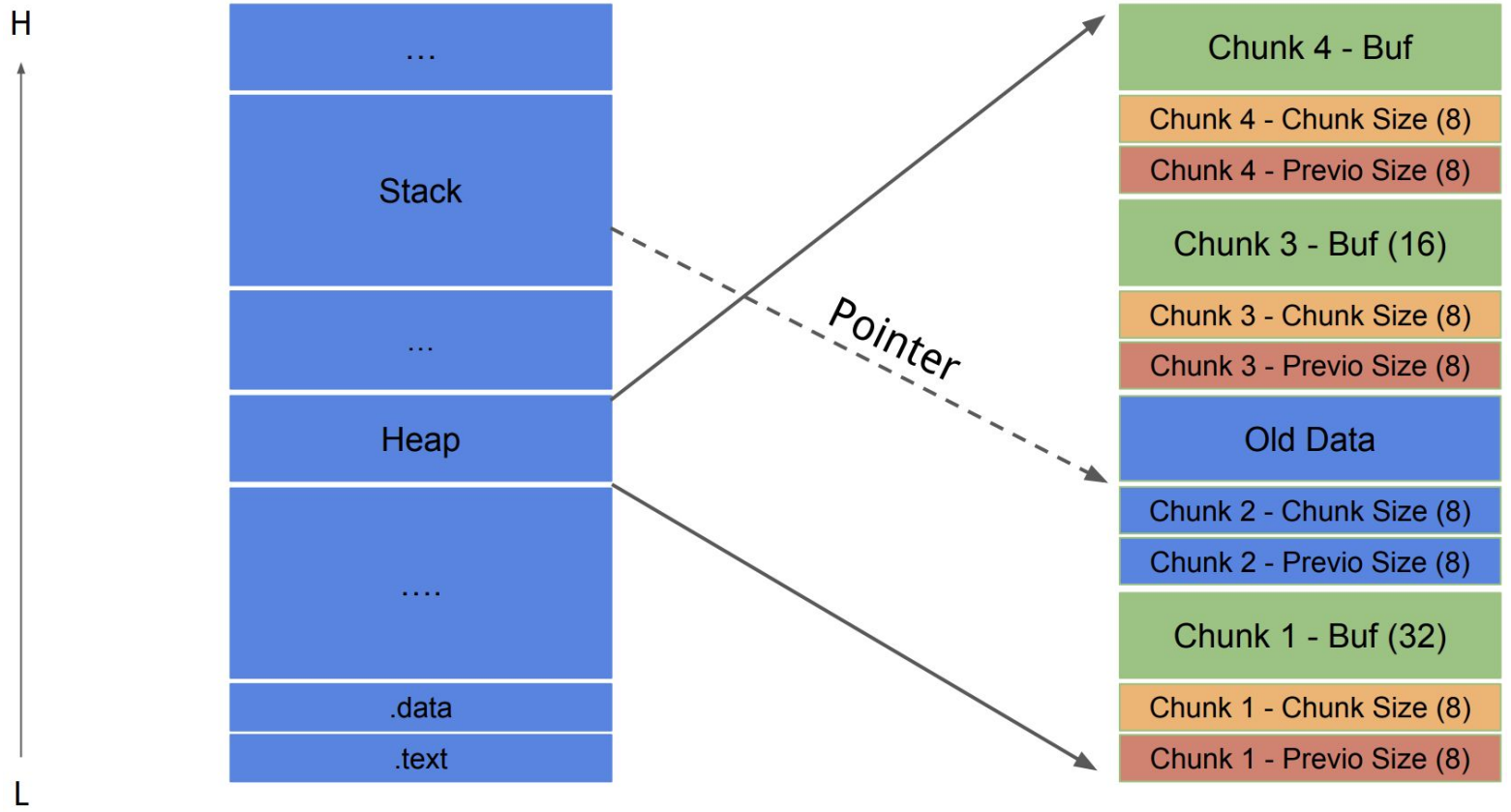
```
python2 -c "print 'a\n' + 'a'*40 + '\x??\x??\x??\\x??\x??\x??\x??'" |
./heapoverflow64
```

Use after free (UAF)

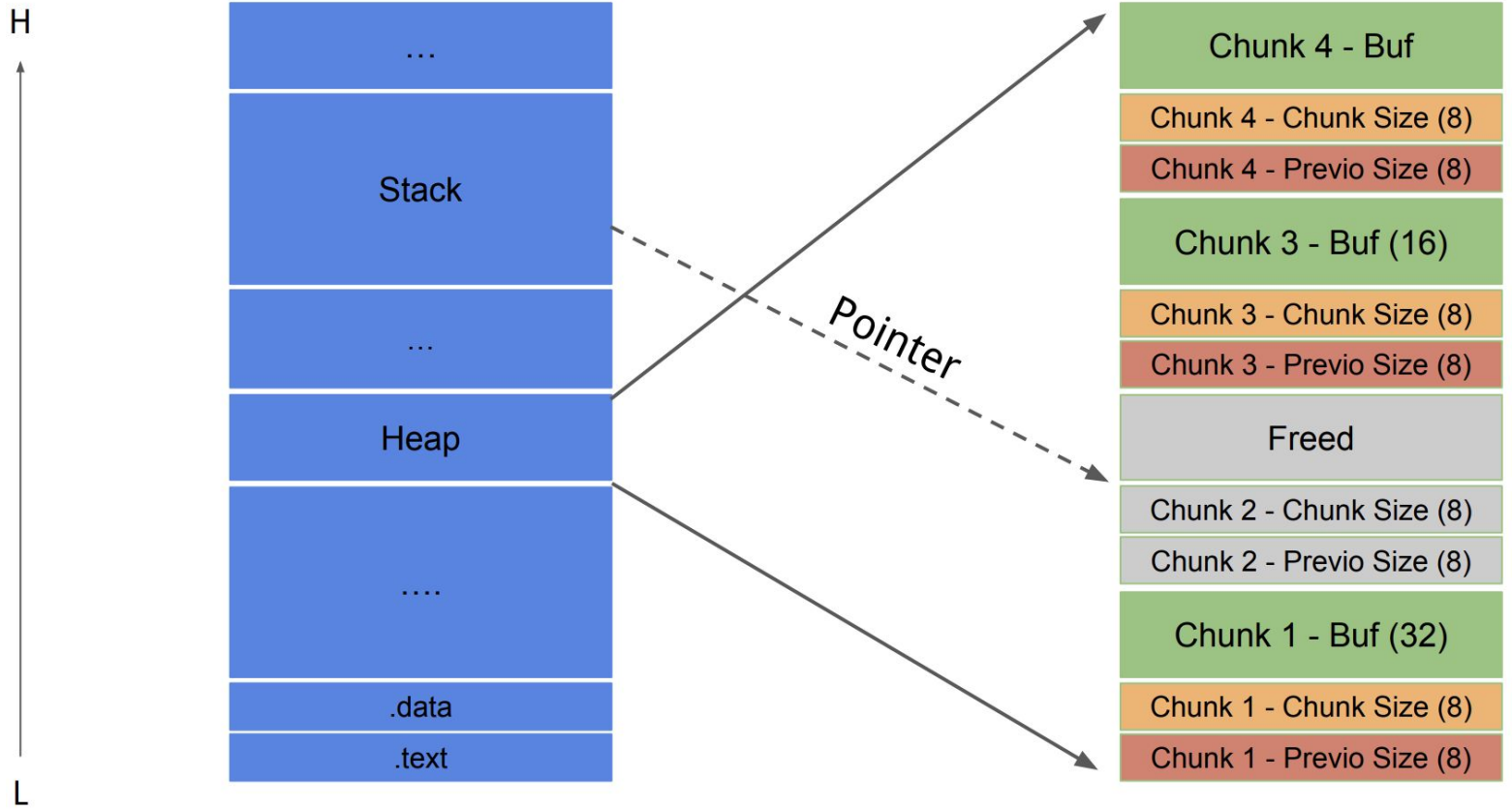
A class of vulnerability where data on the heap is freed, but a leftover reference or ‘dangling pointer’ is used by the code as if the data were still valid.

Most popular in Web Browsers, complex programs

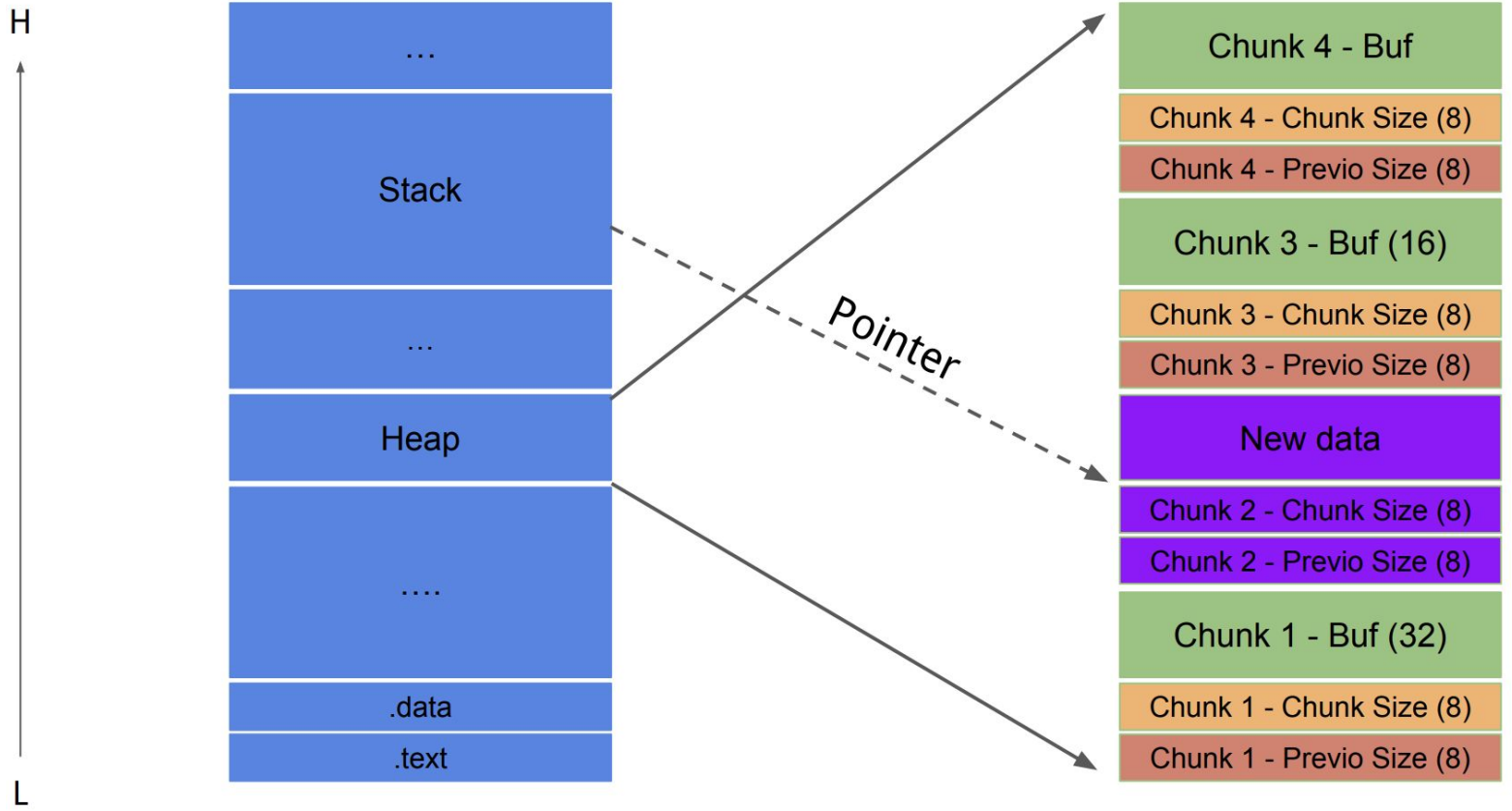
Use after free (UAF)



Use after free (UAF)



Use after free (UAF)



Dangling Pointer

Dangling Pointer

- A left over pointer in your code that references free'd data and is prone to be re-used
- As the memory it's pointing at was freed, there's no guarantees on what data is there now
- Also known as **stale** pointer, **wild** pointer

Exploit UAF

To exploit a UAF, you usually have to allocate a different type of object over the one you just freed

code/heapuaf1 32bit

```
void fly()
{
        printf("Flying ...\n");
}

typedef struct airplane
{
        void (*pfun)();
        char name[20];
} airplane;

typedef struct car
{
        int volume;
        char name[20];
} car;
```

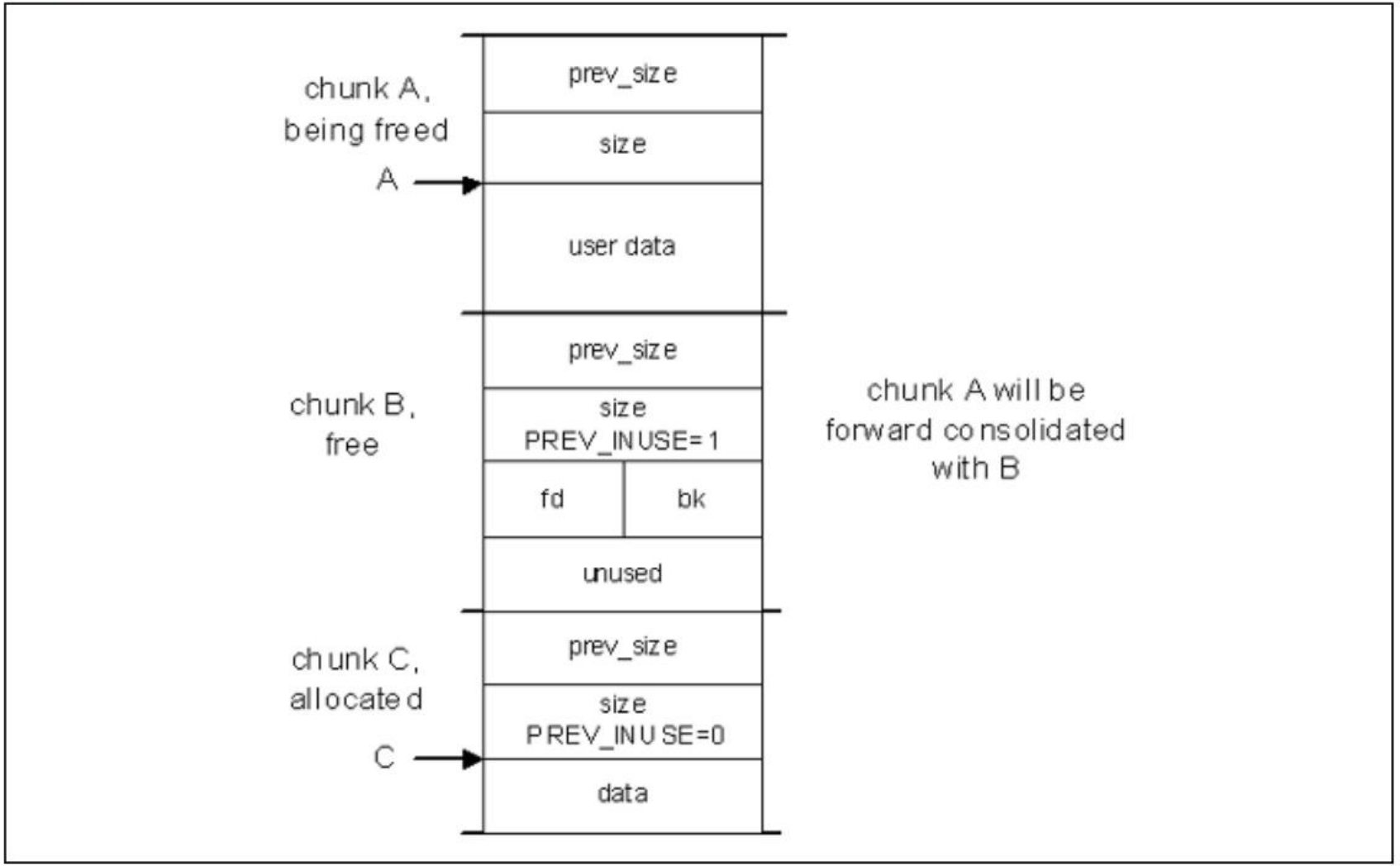
```
int main()
{
        printf("fly() at %p; print_flag() at %u\n", fly,
        (unsigned int)print_flag);

        struct airplane *p = malloc(sizeof(airplane));
        printf("Airplane is at %p\n", p);
        p->pfun = fly;
        p->pfun();
        free(p);

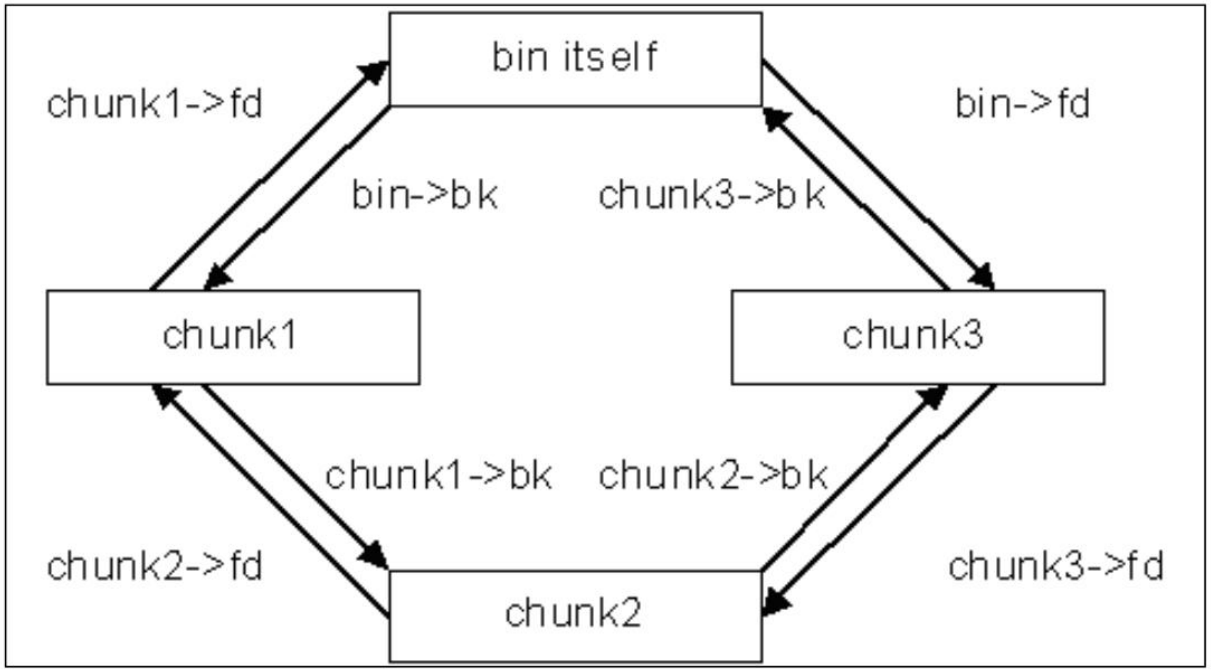
        struct car *p1 = malloc(sizeof(car));
        printf("Car is at %p\n", p1);

        int volume;
        printf("What is the volume of the car?\n");
        scanf("%u", &volume);
        p1->volume = volume;

        p->pfun();
        free(p);
        return 0;
}
```


chunk A,
being freed
A
prev_size
size
user data
chunk B,
free
prev_size
size
PREV_INUSE=1
fd
bk
unused
chunk A will be
forward consolidated
with B
chunk C,
allocated
C
prev_size
size
PREV_INUSE=0
data

Example Bin with Three Free Chunks



FD and BK are pointers to “next” and “previous” chunks inside a linked list of a bin, ***not adjacent physical chunks***.

Pointers to chunks, physically next to and previous to this one in memory, can be obtained from current chunks by using **size** and **prev_size** offsets.

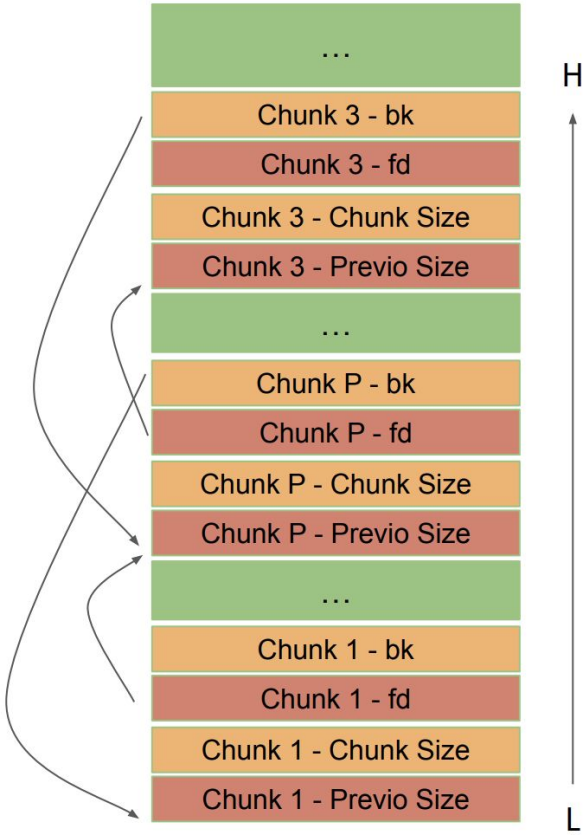
Historical: Pointers to physically next to and previous chunk

```
/* Ptr to next physical malloc_chunk. */
#define next_chunk(p) ((mchunkptr)( ((char*)(p)) + ((p)->size & ~PREV_INUSE) ))


/* Ptr to previous physical malloc_chunk */
#define prev_chunk(p) ((mchunkptr)( ((char*)(p)) - ((p)->prev_size) ))
```

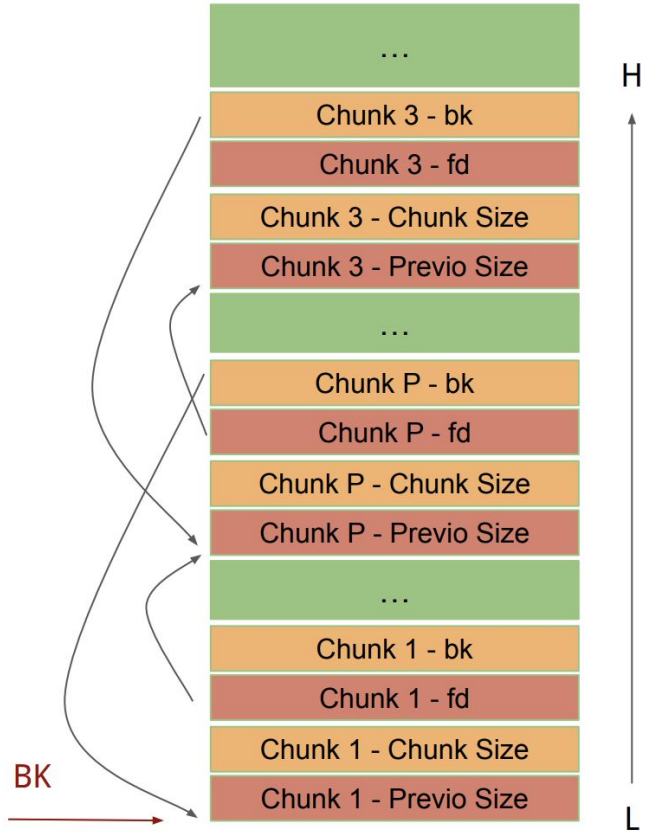
Unlink() a Free Chunk P from the Bin

```
#define unlink( P, BK, FD )
{
        BK = P->bk;
        FD = P->fd;
        FD->bk = BK;
        BK->fd = FD;
}
```



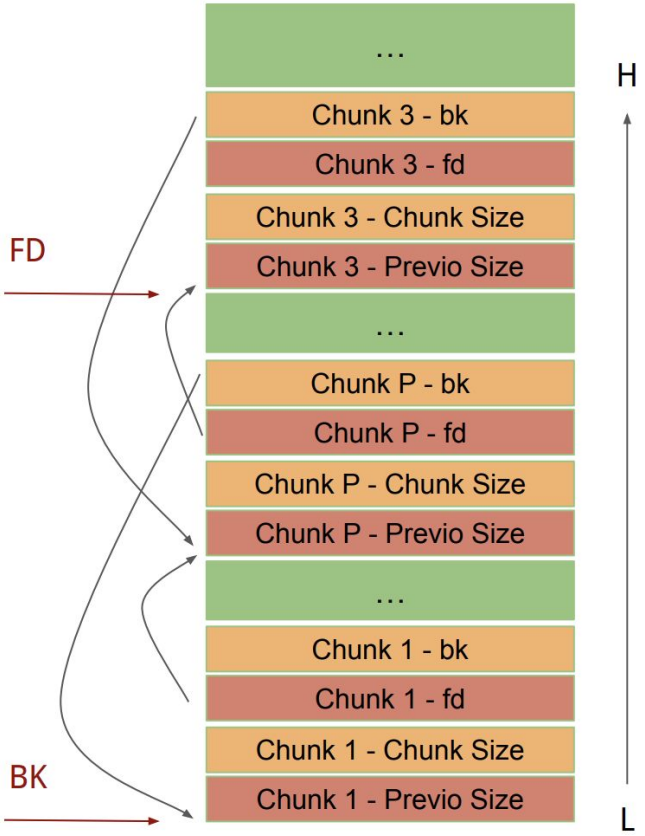
Unlink() a Free Chunk P from the Bin

```
#define unlink( P, BK, FD )
{
        BK = P->bk;
        FD = P->fd;
        FD->bk = BK;
        BK->fd = FD;
}
```



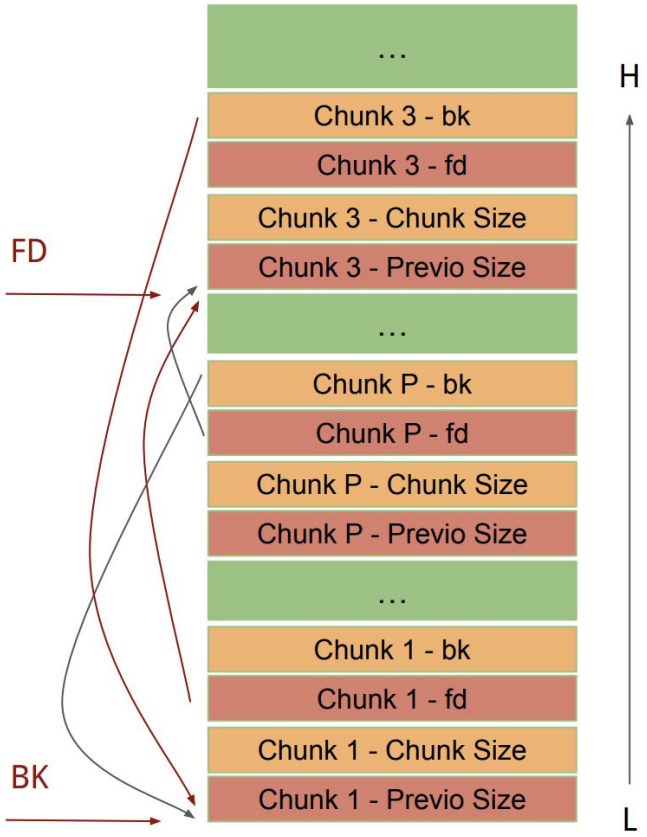
Unlink() a Free Chunk P from the Bin

```
#define unlink( P, BK, FD )
{
        BK = P->bk;
        FD = P->fd;
        FD->bk = BK;
        BK->fd = FD;
}
```



Unlink() a Free Chunk P from the Bin

```
#define unlink( P, BK, FD )
{
        BK = P->bk;
        FD = P->fd;
        FD->bk = BK;
        BK->fd = FD;
}
```



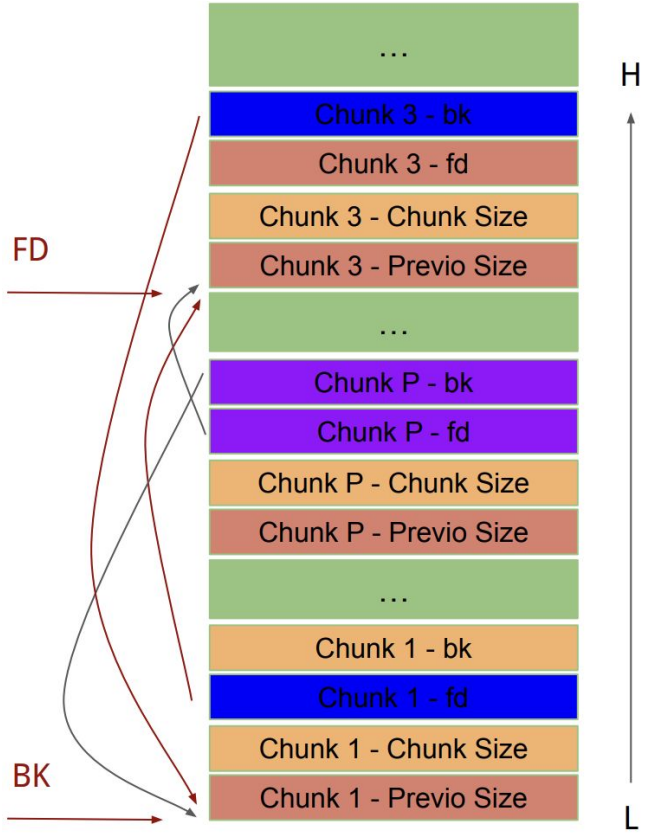
Unlink() from an Attacker's Point of View

```
*(P->fd+12) = P->bk;
// 4 bytes for size, 4 bytes for prev_size and 4 bytes for fd

*(P->bk+8) = P->fd;
// 4 bytes for size, 4 bytes for prev_size
```

Arbitrary write attack?

If an attacker is able to overwrite these two pointers and force the call to unlink(), he can overwrite any memory location.



The free() Algorithm

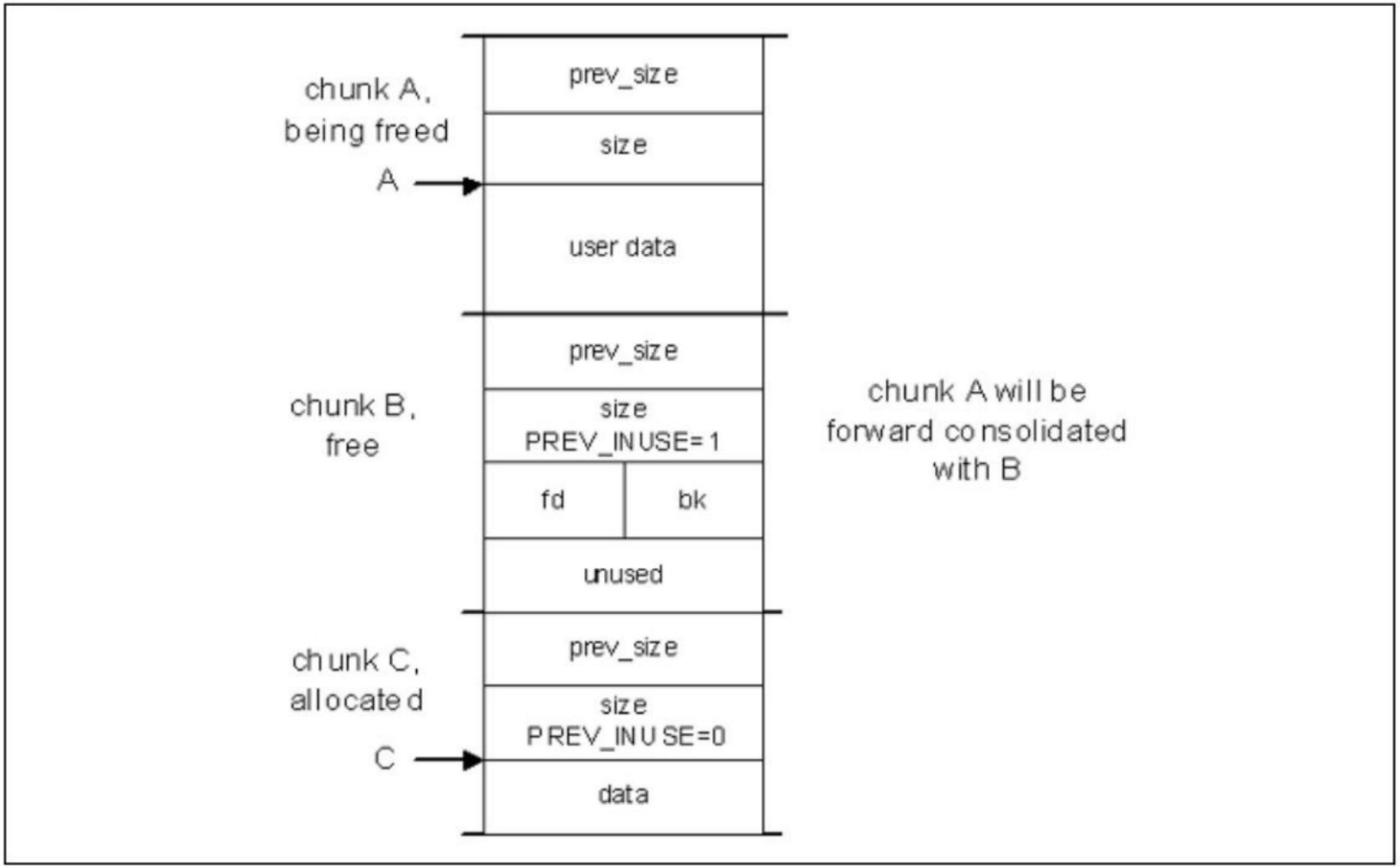
- free(0) has no effect.
- If a returned chunk borders the current high end of memory (wilderness chunk), it is consolidated into the wilderness chunk, and if the total unused topmost memory exceeds the trim threshold, malloc_trim() is called.
- Other chunks are consolidated as they arrive, and placed in corresponding bins.

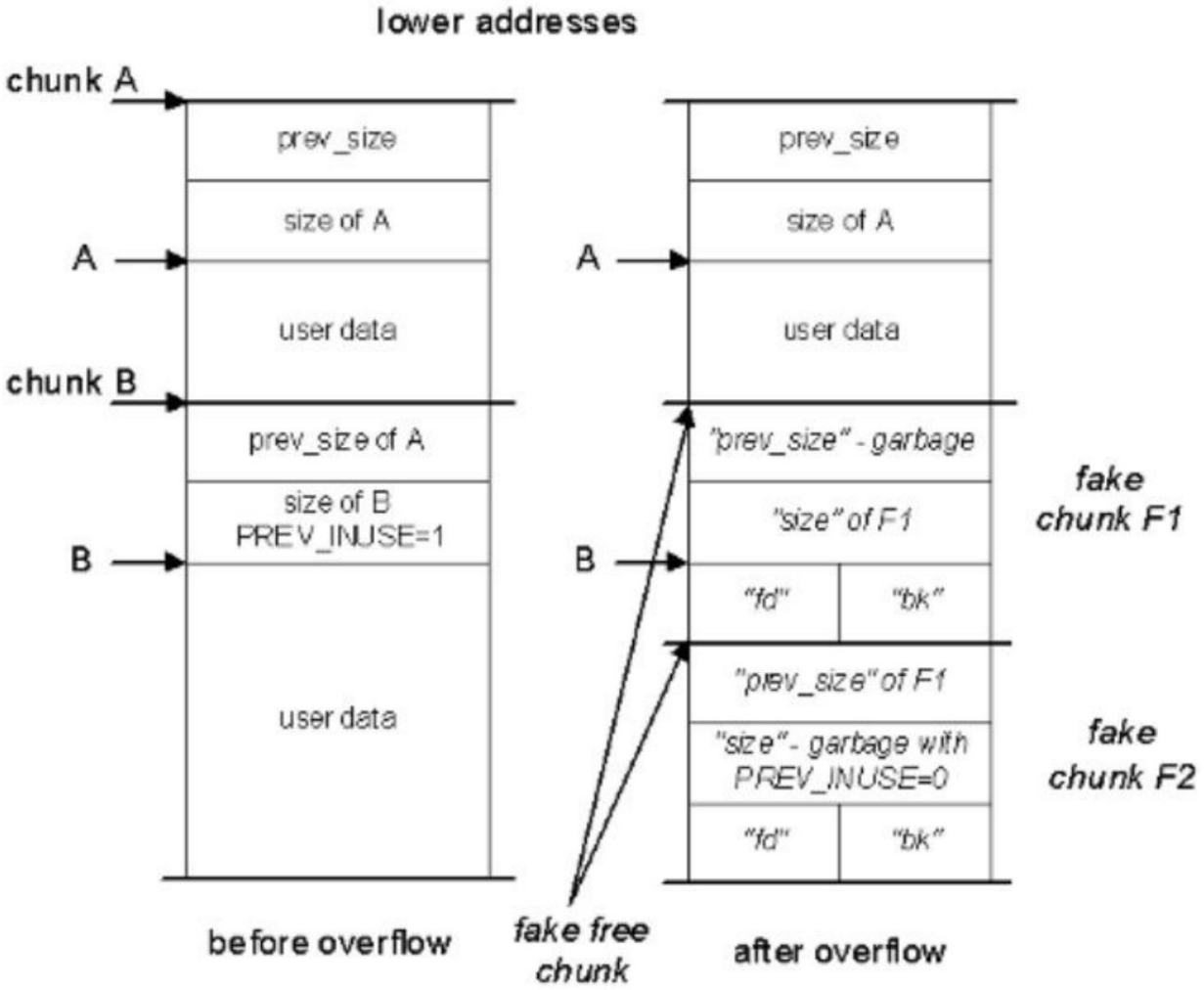
The free() Algorithm - last case

- If no adjacent chunks are free, then the freed chunk is simply linked into corresponding with bin via frontlink().

- If the chunk next in memory to the freed one is free and if this next chunk borders on wilderness, then both are consolidated with the wilderness chunk.

- If not, and the previous or next chunk in memory is free and they are not part of a most recently split chunk (this splitting is part of malloc() behavior and is not significant to us here), they are taken off their bins via unlink(). Then they are merged (through forward or backward consolidation) with the chunk being freed and placed into a new bin according to the resulting size using frontlink(). If any of them are part of the most recently split chunk, they are merged with this chunk and kept out of bins. This last bit is used to make certain operations faster.


chunk A,
being freed
A
prev_size
size
user data
chunk B,
free
prev_size
size
PREV_INUSE=1
fd
bk
unused
chunk C,
allocated
C
prev_size
size
PREV_INUSE=0
data
chunk A will be
forward consolidated
with B



1. Overwrite A and B
2. Create a fake chunk F1 and F2, so that when free(A), unlink(F1) is also called.
3. F1->FD has the address we want to overwrite and F1->BK has the data we want to overwrite